

ארגון ותכנות המחשב

תרגיל בית 4 (רטוב)

המתרגל האחראי על התרגיל: אראג מחיטריאן.

הנחיות:

- שאלות על התרגיל ב- Piazza בלבד.

- ההגשה בזוגות.
- על כל יום איחור או חלק ממנו, שאינו באישור מראש, יורדו 5 נקודות.
 - ניתן לאחר ב-3 ימים לכל היותר.
 - הגשות באיחור יתבצעו דרך אתר הקורס.
- את התרגיל יש להגיש באתר הקורס כקובץ zip.
- תיקונים לתרגיל, אם יהיו, יופיעו ממורקרים.

הערה: ניתן למצוא דוגמאות output בסיסיות בסוף המסמך.

כתיבת דיבאגר

מבוא

שרלוק הולמס ממשיך לחקור פונקציות חשודות בקבצי ריצה, אך נמאס לו להזין כתובות מסובכות ידנית, הוא רוצה הפעם שיהיה אפשר פשוט להכניס את שם הפונקציה והתוכנית החוקרת שלו תמצא תמצא את הכתובת שלה לבד. הוא יודע שלשם כך התוכנית תצטרך למצוא את הפונקציה בטבלת הסמלים של קובץ הריצה (.symtab). אחרי שתמצא את הכתובת, התוכנית תוכל לבצע מעקב אחרי ערכי הפרמטרים שהפונקציה מקבלת, אחרי ערכי החזרה שלה ובנוסף גם תעקוב אחרי קריאות רקורסיביות שהפונקציה מבצעת.

ארגומנטים (Command Line Arguments)

התוכנית שלכם (prf) תקבל את הארגומנטים הבאים לפי הסדר:

1. `sym_name` – מחרוזת של שם הפונקציה החשודה שאחריה יש לעקוב.
2. `number_of_input_params` – מספר הפרמטרים שהפונקציה הנבדקת מקבלת (מספר שלם בין 0 ל-6).
3. `path_to_elf_file` – נתיב לקובץ הריצה (בפורמט ELF64) אותו נרצה לחקור.
4. ארגומנטים אופציונליים נוספים שיש להעביר לקובץ הריצה הנבדק (כלומר ה-`elf_file`).

דוגמת הרצה בטרמינל:

```
./prf function_name 3 path/to/elf_file [target_args...]
```

אחרי מי התוכנית עוקבת?

כל המעקב מתבצע על פונקציה אחת בלבד, שאת שמה תקבלו כארגומנט ראשון. כל הפונקציות ישמרו על קונבנציות System V, אבל לא בהכרח ייכתבו בשפת C (כלומר, ייתכנו טסטים שנכתבו באסמבלי. לא בשפות עיליות אחרות). בנוסף, מובטח לכם כי הפונקציה קיימת, היא מקבלת עד שישה ארגומנטים (יכולה גם לקבל אפס ארגומנטים), אשר כולם `int`, ובנוסף הפונקציה מחזירה `int`.

שלבי ביצוע המעקב

שלב א': פענוח קובץ ה-ELF עבור מציאת כתובת הפונקציה

התוכנית תקרא את קובץ הריצה שסופק לה, בשביל למצוא את כתובת הריצה של הפונקציה. בעת מציאת הפונקציה יודפס, בראש התוכנית לפני כל הדפסה אחרת:

```
"PRF:: symbol address is <function_address_in_hex>\n"
```

הערה: לכתובת הפונקציה שמודפסת צריכה להיות קידומת של "0x", לדוגמה: 0x405423.

אם הפונקציה לא נמצאה, אז יש להדפיס:

```
"PRF:: symbol not found\n"
```

ולסיים את ריצת התוכנית עם ערך חזרה של 1 מ-`main`. הכל מקרה אחר (הפונקציה כן נמצאה ולכן אפשר

לעשות מעקב) יש לסיים את התוכנית עם ערך חזרה של 0 מ-`main`.

שלב ב': בדיקת ה-Prologue של הפונקציה

התוכנית צריכה להדפיס **פעם אחת** אחרי הדפסת הכתובת של הפונקציה ולפני כל ההדפסות של **שלב ג'**, האם הפונקציה הנבדקת מתחילה בפקודה `push %rbp` או לא. אם כן, יש להדפיס:

```
"PRF:: This function starts by pushing rbp\n"
```

אם לא – לא מדפיסים את השורה וממשיכים הלאה לשלב ג'.

שלב ג': מעקב אחרי קריאות לפונקציה והפרמטרים שהיא מקבלת

בכל פעם שמתבצעת קריאה לפונקציה הנבדקת, עלינו לבדוק מה ערכי הפרמטרים שהועברו אליה. **שימו לב** שיש להבחין בין שני סוגי קריאות:

1. **קריאה ראשונית לפונקציה:** הקריאה הראשונה לפונקציה, המתבצעת **מחוץ** לפונקציה (למשל מתוך `main`). יש להדפיס:

```
"PRF:: run #<call_counter> called with (<param1>, <param2>, ...):"
```

הערה לגבי קריאה ראשונית: הכוונה לקריאה הראשונה לפונקציה מתוך פונקציה אחרת. אם הפונקציה נקראת, מסתיימת, ואז נקראת שוב מתוך פונקציה אחרת, זהו רצף קריאה ראשונית חדש.

2. **קריאה רקורסיבית:** קריאה לפונקציה הנבדקת, אשר מתבצעת **מתוך** קריאה קודמת לפונקציה הנבדקת (הן רקורסיה ישירה והן רקורסיה הדדית). במקרה זה, יש להדפיס:

```
"PRF:: entered recursive call with (<param1>, <param2>, ...)"
```

שימו לב להזחה בגודל של 4 רווחים.

הערה: אם הפונקציה לא מקבלת פרמטרים, אז פשוט מדפיסים אותו הדבר, אבל הסוגריים יהיו ריקים, לדוגמה:

```
"PRF:: run #<call_counter> called with ():"
```

או במקרה של רקורסיה:

```
"PRF:: entered recursive call with ()"
```

שלב ד': הדפסת ערך החזרה

רק כאשר חוזרים מהקריאה החיצונית ביותר לפונקציה, יש להדפיס את ערך החזרה שנשמר ברגיסטר `RAX`:

```
"PRF:: call to function returned with <return_value>"
```

שימו לב להזחה בגודל של 2 רווחים במקרה זה.

הערות

1. אתם מקבלים בשביל נוחות, קובץ `example.c` אשר ישמש אתכם כבסיס לשיעורי בית, לא חייב להשתמש בו, אך מומלץ. כל מה שנמצא ב- `#include` אצלו אמור להיות מספיק בשביל לפתור את התרגיל. בנוסף מומלץ להבין מה הקוד הנתון בקובץ הזה עושה, זה יעזור לכם להבין איך להתחיל את התרגיל בית.
2. שימו לב שהמימוש **אינו** דורש להדפיס את ערכי החזרה של שלבי הביניים ברקורסיה, אלא רק את ערך החזרה הסופי של הקריאה החיצונית ביותר לפונקציה.
3. לסיים את התוכנית תמיד דרך ה-`main`.
4. אפשר להניח כי כל קריאות המערכת יצליחו ואין צורך לבדוק זאת בקוד המוגש.
5. שימו לב להוראות. כל הדפסות התוכנית צריכות להסתיים בירידת שורה ולהתחיל עם `prefix` של: `"PRF::"`

טסטים

חשוב! אם ברצונכם לכתוב תוכניות קלט בשפת C עבור טסטים, יש לקמפל אותם עם דגל -no-pie. רק כך, הפונקציה שאותה אתם רוצים לחקור תקבל כתובת סטטית בזמן הקישור. מה שמאפשר לגלות לאיזה כתובת נטענה הפונקציה שאתם מעוניינים לחקור.

הוראות הגשה לתרגיל הבית

אנא קראו בעיון את החלק הזה ושימו לב שאתם מגישים את מה שצריך לפי ההוראות המופיעות כאן - חבל מאוד שתצטרכו להתעסק בעוד מספר שבועות מעכשיו בערעורים, רק על הגשת הקבצים לא כפי שנתבקשתם.

מה להגיש?

עליכם להגיש קובץ zip יחיד המכיל את כל הקבצים הנחוצים לבניית הפתרון. **אין לשים תיקיות ב-zip.** אנא הגישו קבצים תקינים בלבד. אנא נבנה את התוכנית שלכם באופן הבא:

```
unzip SUBMITTED_ZIP_FILE
gcc -std=c99 *.c -o prf
```

(שימו לב לקמפול לפי c99)

הערות

1. הקלט תקין: התוכנית שלכם תמיד תקבל את מספר הפרמטרים הנדרש בסדר הנכון (כפי שצוין לעיל) והם יהיו בפורמט תקין.
2. ניתן להניח שאין שגיאות מערכת הפעלה.
3. ניתן להניח שאם התהליך (שמקבלים כקלט) רץ בפני עצמו (כלומר, לא בדיבאגר) הוא יסתיים בצורה תקינה ולא מקבל סיגנלים.
4. ניתן להניח שהפונקציה הנבחרת לא תהיה הפונקציה _start.
5. אין לשנות את התנהגות התוכנית המדובגת (שמקבלים כקלט).

בהצלחה!!!

דוגמת הרצה

בתיקייה המכילה את כל קבצי ה-C שכתבתם עבור הדיבאגר, וכן את קובץ הטסט, הריצו את הפקודות הבאות:

```
$ gcc -std=c99 *.c -o prf
$ timeout 20s ./prf foo 1 program1.out
PRF:: symbol address is 0x401106
PRF:: This function starts by pushing rbp
PRF:: run #1 called with (3):
PRF::     entered recursive call with (2)
PRF::     entered recursive call with (1)
PRF::     call to function returned with 6
```

דוגמאות output

דוגמה 1: פונקציה עם `push rbp`

```
PRF:: symbol address is 0x401100
PRF:: This function starts by pushing rbp
PRF:: run #1 called with (5):
PRF::     call to function returned with 10
```

דוגמה 2: פונקציה ללא `push rbp`

```
PRF:: symbol address is 0x401200
PRF:: run #1 called with (10, 20):
PRF::     call to function returned with 30
```

דוגמה 3: מספר קריאות לפונקציה מתוך `main`

```
PRF:: symbol address is 0x401300
PRF:: run #1 called with (2):
PRF::     call to function returned with 4
PRF:: run #2 called with (3):
PRF::     call to function returned with 9
```

דוגמה 4: פלט של רקורסיה הדדית:

קוד הנבדק:

```
int rec_foo(int n) {
    if(n==1) return 0;
    return rec_bar(n-1)/2;
}

int rec_bar(int n) {
    if(n==1) return 1;
    return n+rec_foo(n-1);
}

int main() {
```

```
    rec_foo(5);  
    rec_bar(5);  
    return 0;  
}
```

התוצאה:

```
PRF:: symbol address is 0x401106  
PRF:: This function starts by pushing rbp  
PRF:: run #1 called with (5):  
PRF::     entered recursive call with (3)  
PRF::     entered recursive call with (1)  
PRF::   call to function returned with 2  
PRF:: run #2 called with (4):  
PRF::     entered recursive call with (2)  
PRF::   call to function returned with 1
```

שימו לב להבדל בין הקריאה מתוך `main` לקריאה מתוך `rec_bar`.